Robust Anomaly-Based Insider Threat Detection Using Graph Neural Network

Junchao Xiao, Lin Yang, Fuli Zhong, Xiaolei Wang, Hongbo Chen, and Dongyang Li

***Abstract*—Misuse or malicious access to critical assets of information systems by insiders usually causes significant loss to organizations. The issue of insider threat detection for information systems has received many researchers' attention in both security and data mining fields, and a lot of related research results were presented. However, there are still many challenges in capturing the behavior difference between malicious insiders and normal users accurately, such as lack of labeled insider threats, the subtle and adaptive nature of insider threats, complexity, heterogeneity, sparsity of the underlying data, etc. To detect insider threats with large and complex audit data, a Multi-Edge Weight Relational Graph Neural Network method (MEWRGNN) for robust anomaly detection is proposed in this paper. Unlike most existing approaches, the MEWRGNN adopts several graph neural networks to capture the contextual relationship of user behaviors over a period of time, which is a critical factor for achieving accurate anomaly identification. The MEWRGNN achieves a certain degree of interpretability through ranking the contribution of different edge-representation features. Evaluation experimental results demonstrate that the MEWRGNN can learn a model from limited sample data sets, and achieve quick and accurate insider threat detection performance. In addition, other feature ranking results allow providing security analysts with understandable insights for investigating the detected insider threats.**

***Index Terms*—Anomaly detection, insider threat, graph neural network.**

I. Introduction

INSIDER threat is known as a kind of malicious threat to the information security of companies or organizations. **Insider Threat** is often defined as a malicious insider who intentionally exploits his or her privileged access to the organization's network, system, and/or data, taking actions in a manner that negatively affected the confidentiality, integrity or availability of the organization's information or information system infrastructures [1]. Insider threats can also be non-malicious [2]. The misuse or malicious behavior from someone in the organization usually causes damage to confidential or commercially valuable information, or even results in the sabotage of computer systems. Insiders often have authorized access to the organization's information systems and detailed knowledge about the network. Threats caused by insiders are usually challenging to detect. Cybersecurity report [3] claimed that more than half of organizations have experienced insider threats in recent years. Insider threat attacks account for a quarter of all types of organizing attacks. This number is still rising [3]. In the meantime, some discovered insider betrayals may sell secrets, customer information, intellectual property, and other important privacy data [4]. Network information system is a critical part of an organization to maintain the efficient operations. Most sensitive data such as employee information, intellectual property and transaction records are stored in the information systems which malicious insiders may easily access. The insider threat problem required to be well addressed to ensure the security of information systems in organizations or companies.

Insider threats are different from external threats, they mainly come from the organization or company, and generally have the characteristics of high risk, concealment and diversity. (i) High risk: Insider threats are more harmful than external threats because adversaries have the internal knowledge of the organization and are more likely to gain access to the organization's core assets (such as intellectual property and user data), and cause considerable losses to the organization's core assets [5]. (ii) Concealment: The adversary comes from organization, and the insider threat often has strong camouflage, which will lead to quite a challenge for its detection by existing security mechanisms. Internal attacks do not need to pass through firewalls and other devices. Thus they can easily evade the detection of traditional external security devices. Internal attacks often occur during working hours. The mixture of attack data with normal data increases the difficulty of malicious insider data mining. Additionally, internal adversaries with knowledge of organizational security defense are more likely to evade security detection [5], [6]. (iii) Diversity: In the Internet era, the organization's core assets and business are highly informatized, resulting in a lower threshold for internal attacks and diversification of attack elements. From the initial computer login users to employees, contractors, partners and service providers, adversaries can use logic bombs to paralyze the system, steal intellectual property rights, or

Manuscript received 29 October 2021; revised 15 March 2022, 24 June 2022, and 12 November 2022; accepted 13 November 2022. Date of publication 16 November 2022; date of current version 9 October 2023. This work was supported by the National Natural Science Foundation of China (Grant No. 62102466), and the Fundamental Research Funds for the Central Universities, Sun Yat-sen University (Grant No. 22qntd1602). The associate editor coordinating the review of this article and approving it for publication was S. Kanhere. *(Corresponding authors: Fuli Zhong; Hongbo Chen.)*

Junchao Xiao, Fuli Zhong, and Hongbo Chen are with the School of Systems Science and Engineering, Sun Yat-sen University, Guangzhou 510006, China (e-mail: xiaojch3@mail2.sysu.edu.cn; zhongfli@mail.sysu.edu.cn; chenhongbo@mail.sysu.edu.cn).

Lin Yang and Xiaolei Wang are with the National Key Laboratory of Science and Technology on Information System Security, Institute of System Engineering, Chinese Academy of Military Science, Beijing 100039, China.

Dongyang Li is with the Command and Control Engineering College, Army Engineering University of PLA, Nanjing 211101, China.

Digital Object Identifier 10.1109/TNSM.2022.3222635

take the advantages of their positions to carry out electronic fraud [5].

CERT Insider Threat Center classified insider threats into two categories. One is the intentional behavior, such as information system sabotage, intellectual property theft and information technology fraud; The other is the negligence of employees in using authorized resources, which leads to information leakage [4]. This paper focuses on detecting the first category, i.e., intentionally malicious behavior detection. Based on the works [7] and [4], insider threats of intentionally malicious behavior are mainly divided into four categories, they are sabotage, theft, fraud and espionage. Sabotage is the act of malicious insider employees deliberately damage or destroy the organization's information system [8]. Theft refers to the theft of intellectual property. Insiders send the intellectual property to a new employer or a competing organization for the profit purpose [8]. Fraud refers to internal personnel who have the financial authority to abuse organizational finances [9]. Espionage is about a malicious insider who downloads corporate information in a planned and organized manner to obtain strategic economic, military, or other benefits [10], [11].

Insider threats detection mainly faces two challenges: (i) In most organizations, malicious behaviors of insiders occur infrequently, the available records of malicious behaviors are limited [12], so extracting suitable features from the limited malicious behavior data becomes one of the challenges. (ii) Mass data of different types, such as network traffic, website and file access records, mail receiving and sending records, need to be investigated and processed [13]. How to fuse such mass data to extract the complex relationship is another challenge. Most existing approaches [13], [14], [15], [16], [17] usually adopt rule-based or statistic-based techniques which are hard to solve these challenges very well. Rule-based methods are often valid for certain specific dataset types. While statistic-based methods usually involve feature engineering, leading to delay in insider threat detection. This paper uses the CMU CERT r6.2 insider threat dataset with a large amount of normal behavior data and a small amount of malicious behavior data which is consistent with real-world situations. we introduce the Graph Neural Network (GNN) method to capture the contextual relationship between users' different behaviors over time to ensure that the proposed method can deal with various datasets and reduce insider threat detection time delay. A Multi-Edge Weight Relational Graph Neural Network model (MEWRGNN) is proposed to improve the detection accuracy, efficiency, and interpretability. The MEWRGNN method employs Relational Graph Convolutional Network (R-GCN) to process graphs with multi-edge weights. Graph Convolutional Network (GCN) is applied to extract relational features between nodes. CAN-Graph Attention Networks (CAN-GAT) is utilized to discover critical edges in graphs. The MEWRGNN method can enhance the interpretability of the model by locating the features that contribute the most to the model performance in the multi-edge weight graph. The model's multi-feature fusion capability is improved by combining several GNNs and a multi-edge weight graph. The proposed approach is mainly aimed at assisting the organization's information system administrators in discovering potential insider threats.

The contributions of this paper are summarized as follows.

- Inspired by graph neural network, a preprocessing method is proposed to transform the user behavior log with time-series features into a graph structure to capture the contextual relationship of user behavior over a period of time. In the graph, nodes and edges preserve the context of each behavior log, which can help improve the detection accuracy.
- The combined graph neural networks method is proposed to extract different user behavior features. Based on these features, the proposed MEWRGNN model can accurately detect insider threats with certain interpretability.
- The proposed MEWRGNN is evaluated with the CERT dataset, and multiple experiments have been conducted for performance tests and comparison. The obtained results show that MEWRGNN outperforms several baseline methods in terms of accuracy and efficiency. Additionally, based on the edge weight values, the simulation experiment with cutting different types of edges that the weight given by MEWRGNN can guide the feature selection and provide security analysts with interpretable results.

The remainder of this paper is organized as follows. In Section II, insider threat research work is introduced. The preliminaries of this work, the preprocessing process of input data sample, and related theory of GNNs are presented in Section III. In Section IV, the MEWRGNN approach is explained in detail. Evaluation results are shown in Section V. Followed by the conclusion in Section VI.

II. Related Work

In the past few years, researchers have developed a variety of insider threat detection methods [6], [8], [18]. Liu et al. [18] introduced the organization's insider threats in detail as well as various types of threats. Homoliak et al. [8] chose a new classification method to study insider threats. Yuan and Wu [6] reviewed the advantages and challenges of insider threat detection using deep learning technique. Representative insider threat detection methods mainly include psychology-based methods, machine learning-based methods and graph-based methods [11].

Some security researchers attempt to solve the insider threat problem from the psychological perspective [19], [20]. Greitzer and Hohimer [19] analyzed human behavior data based on psychological theory, and proposed a predictive model to detect insider threats. Brdiczka et al. [20] explored the method of psychological context, and constructed a graph learning model to predict threat behaviors of internal personnel in the organization. Analyzing insider threats with psychological theory can accurately locate threats based on human psychology. The above method confirms that the combination of each employee's psychological test form and appropriate psychological theory can help to accurately detect insider threats. However, this method requires a large amount of expert knowledge and prior knowledge to provide decision

support for detection. Data characteristics analysis should be carried out in advance for each dataset.

Academic and industrial communities have recently focused on machine learning and deep learning techniques for anomaly-based insider threat detection. The learning model can extract the user's normal behavior characteristics, and identify abnormal behaviors based on the degree of their deviation from the normal behaviors. If an abnormal alarm is triggered, it may indicate that the user's behavior has changed significantly. It can be treated as an early warning of insider threats. Parveen et al. [21] abstracted the insider threat problem as a streaming data analysis problem, and used the one-class support vector machine (SVM) to deal with drifting features. This method can effectively solve the problem of insufficient labeled abnormal data. Insider threat detection methods using Recurrent Neural Network [22], Convolutional Neural Network [23] and other deep learning models were reported in recent years. Chattopadhyay et al. [24] applied statistical single-day features to construct a time series vector, and used a deep autoencoder neural network to implement threat detection to obtain better results than multilayer perceptron. These methods take advantage of the powerful ability of deep learning technique to capture abstract features. Stream online learning [25], [26] is a method of insider threat detection based on machine learning and deep learning as well. It attempts to capture an abstract representation of the user behavior. Tuor et al. [25] chose deep neural networks to build different models for each user and organization for anomalous network activity detection. Böse et al. [26] proposed the real-time anomaly detection in streaming heterogeneity for the streaming anomaly detection at scale. Parveen and Thuraisingham [27] designed an incremental learning model of streaming data for insider threat detection via using unsupervised learning algorithm. Detection methods based on machine learning and deep learning have a certain ability to detect unknown threats. Machine learning-based methods can perform an accurate insider threat detection as long as they collect enough data and extract suitable features. It is difficult to judge whether the behavior record is normal or not based on only a small number of behavior records in the threat detection. To improve the detection accuracy of the detection model, multiple behavior records are required for each sample that is input into the model for detection. However, when enough behavioral records have been collected to make up one input sample, the malicious behavior may have passed hours or even days [15].

Researches on insider threat detection based on graph model were presented in the past few years [17], [20]. Liu et al. [17] studied the log2vec which is a heterogeneous graph embedding based modularized method for cyber threats detection in enterprise, and applied the similarity feature of each user behavior log as one of the basis to construct graph structure samples for abnormal user detection. Graph model based method can use features like sequential relationships among log entries to improve the threat detection accuracy [17], but often requires too much prior knowledge and a relatively high amount of computation which leads to threat detection delay.

In recent years, Graph Neural Network (GNN) technique has been widely used in the knowledge graph, computer vision, chemistry, and so forth [28], [29]. For its superior performance in these application fields, GNN technique was applied to inside threat detection [30]. Jiang et al. [30] constructed graph format model with users' behaviors and their connection relationships, and trained the GCN based anomaly detection model with the formed input graphs, and then detected the insider threats with the obtained detection model. In this method, a large amount of data is input into the model directly for the training task, which will result in much more training time cost and higher hardware conditions requirement. As distinct from previous works in the literature, the MEWRGNN model is proposed to detect insider threats in this paper. The presented model can automatically discover the relationship between internal personnel through the constructed graph data, extract detection features, and reduce the amount of data required for each detection. It helps to improve the model's versatility and decreases the delay of threat detection.

III. Preliminaries

In this section, the CMU CERT datasets used in this research is introduced, then the methods of dataset preprocessing and how the user behavior logs are transformed into graphs are presented, followed by the introduction of related theory of GNN.

A. CERT Insider Threat Dataset

The synthetic CMU CERT datasets [31] were used to evaluate methods in many recent studies. CERT dataset contains 5 log files which record the computer-based operations of all employees in a simulated organization. Its data generation process reflects the practical constraints. The description of log files are as follows.

- Logon.csv – The file records the login and logout operations of all employees.
- Email.csv – The file records the sending and receiving operations and the text content of emails.
- Http.csv – The file records the browser's upload, download, update and text content.
- File.csv – The file records file opening, writing, copying and deleting operations.
- Decive.csv – The file records the connection and disconnection of the removable drive.

So far, several versions of the CERT Insider Threat Dataset have been released, among which the version r6.2 is used more widely. Version r6.2 records the activities of 5 malicious insiders and 3995 normal users from January 2010 to June 2011. The average activity log for each user is about 40,000. The following five insider threat scenarios are involved with the CERT version r6.2.

- User ACM2278, who has never used a removable drive before, assessed the system after work, used the removable drive and uploaded data to wikileaks.org.
- User CMP2946, browsed job search websites, searched for jobs from competing companies, and used a thumb drive to steal company data before leaving the company.

TABLE I
THE NUMBER OF NORMAL AND MALICIOUS ACTIVITIES OF EACH INSIDER

User	ACM2278	CMP2946	PLJ1771	CDE1846	MBG3183
Normal activity	30,946	54,129	20,737	33,024	42,556
Malicious activity	601	8,060	230	4,838	4

- System administrator PLJ1771 felt dissatisfied. He downloaded the keylogger and uploaded it to the supervisor's computer using a thumb drive. The next day, he used his supervisor account to log into the system and sent out many alarming emails, causing panic and people to leave the organization immediately.
- User CDE1846, logged into other users' computers, searched for interesting files and sent these files to their home email addresses.
- User MBG3183, as a laid-off worker, uploaded documents to Dropbox for his profit.

Table I shows the statistics on the number of activities of the insiders, including their normal activity and malicious activity. One can find that the insider activity data is imbalanced.

B. Dataset Preprocessing

The activity logs of 4000 users are scattered into five files, i.e., Logon.csv, email.csv, http.csv, file.csv, and decive.csv. To facilitate the analysis of user activities, each user's activity log is extracted from the five files to form a single user's activity log. Take user ACM2278 as an example, one extracts the activity log of user ACM2278 from the five files and arranges them in chronological order, finally gets 31547 log records. The log format of the synthesized ACM2278 is shown in Table II. Other user logs are obtained in the similar way. Related description of each column of the log format is presented in the Appendix of this paper.

Most user behavior activity logs extracted from the five files are recorded in text format. It is necessary to convert the logs of text format into those of digital format so that the GNN framework can process these logs data properly.

As shown in Table II, the date field records the date and time when the activity occurred. To facilitate computer processing, the date field is converted into a Unix timestamp [32]. The User and PC fields are deleted so that the remaining data fields mainly include attribute parameters of user behavior activity, which helps the GNN model capture user activities and maintain its generality. The size field is already of numeric representation and does not need to be converted any more.

The activity field has only a dozen of different activities. It is suitable for using a factorization method [33] to obtain the numeric representation of corresponding activities. The formula is written as

$$\{1, 2, \ldots, N\} = F(data(activity)) \tag{1}$$

where *data*(*activity*) is the text description form of the activity field of the activity log, the text is converted into an integer of numeric representation through factorization method denoted by $F(\cdot)$, N represents the number of different activities generated by the user. The same activities of different users are converted into exact numeric representation form. Fig. 1 shows the process of converting the activity field of ACM2278 log into the numeric representation form. Other user logs are converted with the similar method.

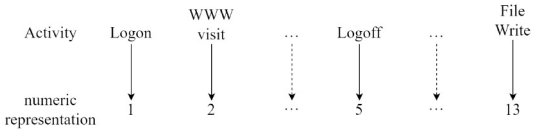


Fig. 1. The process of converting the Activity field of log of ACM2278 into a numeric representation.

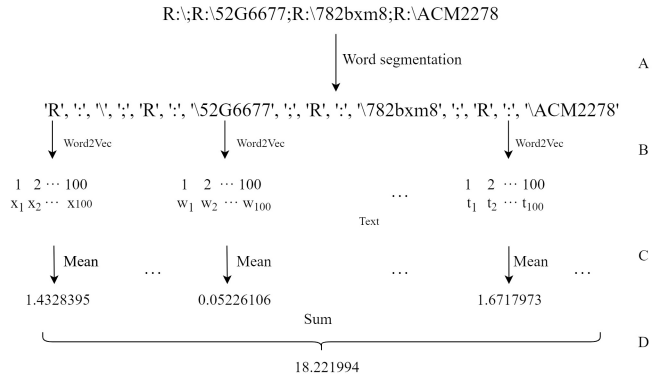


Fig. 2. The process of converting a storage address recorded in text form into a numeric representation.

Other fields of the log are of text representations with many words. To convert these texts into a numeric representation form, Word2Vec [34], [35], [36], [37] is applied. Word2Vec is a group of correlation models for word-vectors generation, in which the models are composed of shallow neural networks and are trained for reconstruction of linguistic word text. After well training, Word2Vec model can map each word to related numerical vector, and help to capture the relation between words. As an unsupervised learning method, Word2Vec is widely used to measure the correlation between words in the natural language processing research field. It is also suitable for achieving the numeric representation of some fields of activity log. Since the "File_tree", "Attachments" and "Filename" fields store file addresses. To use Word2Vec method, these file address texts will be word segmented, written as

$$\{W_1, W_2, \ldots, W_N\} = D(S_1, S_2, \ldots, S_L) \tag{2}$$

where $S_i (i = 1, 2, \ldots, L)$ is the i-th storage address record, $W_i (i = 1, 2, \ldots, N)$ is the i-th separated word from S_i, and $D(\cdot)$ is the word segmentation program. The gensim toolkit [38] is used to train Word2Vec vectors for the separated words $\{W_1, W_2, \ldots, W_N\}$, written as

$$\left\{\hat{W}_1, \hat{W}_2, \ldots, \hat{W}_N\right\} = M_f(\{W_1, W_2, \ldots, W_N\}) \tag{3}$$

where M_f represents the trained Word2Vec model. Base on experience, one defines a 100-dimensional vector to represents a word. To reduce the dimensionality of the activity log, we performed dimensionality reduction for each word. The dimension reduction process consists of four steps, i.e., steps A, B, C, and D in Fig. 2. In step A, file addresses are separated into words by specific symbols such as ',' , ':' and ';'. In step B, each word is mapped to a representation of 100 numbers (for

TABLE II
THE ACTIVITY LOG FORMAT OF ACM2278

Date	User	PC	File_tree	Activity	TO	CC	BCC	From	Size	Attachments	Content	Filename	To_removable_media	From_removable_media	URL
01/04/2010 07:30:00	ACM2278	PC-8431		Logon											
01/04/2010 07:40:08	ACM2278	PC-8431		WWW Visit							{text}				{link}
									...						
01/04/2010 11:19:17	ACM2278	PC-8431		Send		{Email address}			1986983	{Storage address}	{text}				
									...						
01/04/2010 11:42:32	ACM2278	PC-8431		View		{Email address}			37181	{Storage address}	{text}				
									...						
08/13/2010 01:14:26	ACM2278	PC-8431	{Storage address}	Connect											
08/13/2010 01:16:10	ACM2278	PC-8431		File Delete							{text}	{Storage address}	FALSE	TRUE	
									...						
08/13/2010 02:35:54	ACM2278	PC-8431		Logon											

TABLE III
THE NUMERIC REPRESENTED ACTIVITY LOG FORMAT OF ACM2278

Timestamp	File_tree	Activity	TO	CC	BCC	From	Size	Attachments	Filename	To_removable_media	From_removable_media	URL	Content-1	Content-2	...	Content-100
1262561400	0	1	0	0	0	0	0	0	0	0	0	0	0	0	...	0
1262562008	0	2	0	0	0	0	0	0	0	0	0	2.6511736	0.07509	0.10446	...	0.04685
									...							
1262575157	0	3	6.94793	0	0	1.79440	1986983	15.01007	0	0	0	0	0.07693	0.04685	...	0.07677
									...							
1262576552	0	4	1.79440	0	0	2.18775	37181	0	0	0	0	0	0	0.08770	...	0.09787
									...							
1281633266	18.22199	9	0	0	0	0	0	0	0	0	0	0	0	0	...	0
1281633370	0	10	0	0	0	0	0	0	2.74980	2	1	0	0	0.09879	...	0.08535
									...							
1281638154	0	5	0	0	0	0	0	0	0	0	0	0	0	0	...	0

TABLE IV
THE NORMALIZED ACTIVITY LOG FORMAT OF ACM2278

Timestamp	File_tree	Activity	TO	CC	BCC	From	Size	Attachments	Filename	To_removable_media	From_removable_media	URL	Content-1	Content-2	...	Content-100
0.983284524295050	0	0.07692	0	0	0	0	0	0	0	0	0	0	0	0	...	0
0.983284997806272	0	0.15384	0	0	0	0	0	0	0	0	0	0.99680	0.07509	0.10446	...	0.04685
									...							
0.983295238265239	0	0.23076	0.51315	0	0	0.73138	0.18112	0.20016	0	0	0	0	0.07693	0.04685	...	0.07677
									...							
0.983296324693124	0	0.30769	0.13252	0	0	0.89171	0.00338	0	0	0	0	0	0	0.08770	...	0.09787
									...							
0.998137719305787	1	0.69230	0	0	0	0	0	0	0	0	0	0	0	0	...	0
0.998137800301128	0	0.76923	0	0	0	0	0	0	0.85214	1	0.5	0	0	0.09879	...	0.08535
									...							
0.998141526086792	0	0.38461	0	0	0	0	0	0	0	0	0	0	0	0	...	0

example, x_1, $x_2, \ldots, x_{100}$ represents 'R') through Word2Vec method. In step C, the arithmetic mean of the 100 numbers representing the word is calculated, and then used to represent the word. In step D, the sum of the numbers representation of each word obtained in step C is calculated. Thus the numeric representation of the file address is obtained.

The e-mail addresses stored in the "TO", "CC", "BCC" and "From" fields, and the Web addresses stored in the "URL" field are converted into their numeric representations in a similar way to the file addresses. The word segmentation symbols of the e-mails are '@' and ';', and the word segmentation symbol of the URL is ':'.

The "Content" field stores a large number of words. The pre-trained Word2Vec model (wiki-news-300d-1M) proposed by Mikolov et al. [39] is used to convert the text in the "Content" field to its numeric representation. Pre-trained Word2Vec model trained with one million words, including Wikipedia 2017, UMBC webbase corpus and statmt.org news dataset. Same as other fields for storing text formats, if the dimension of the activity log is too high, it will increase the computational cost of the model in the training and detection phases. To keep the dimension of the activity log within a reasonable range and reduce the computational cost, aggregation function is used to calculate the aggregation value of converted numbers. Based on experience, one performs sensitivity analysis on arithmatic average aggregation function with the first 50 to 200 words of the "Content" field of the activity log to search for the suitable ones. Based on the results in Table VII, the first 100 words are finally selected to represent the feature of "Content" field. Another two aggragation functions, arithmatic median and mode (mode is the value that appears most frequently in a set of data.), are also applied to calculate aggregation values with the numerical representation of each word (step C of Fig. 2) for comparison analysis. The experimental results show that the obtained detection accuracy and some other detection evaluating indicators results when using the aggregation function median and the arithmetic average, are the same. For the mode function, the obtained results (shown in Table VIII) are not so good as the above ones when detecting user ACM2278. Furthermore, the arithmetic average calculation is fast and can facilitate further analysis [40].

The "To_removable_media" and "From_removable" fields stores Boolean variables, and the transition rule is TRUE to 1, and FALSE to 2. Other default values in the activity log are filled with 0. Table III shows the result of the numeric representation of the activity log of ACM2278 listed in Table II. In Table III, the "date" field is replaced with "timestamp" field and the "user" and "PC" fields are deleted.

To unify the values of different fields and accelerate the convergence speed of the GNN model in the training phase, some fields in Table III need to be normalized. According to Table III, the 13 fields from "timestamp" to "URL" all need to be normalized (Min-Max Normalization [41]), and note that the numeric representations of the "Content" field by Word2Vec have been normalized. As an example for explanation, the normalized activity log format of ACM2278 is shown in Table IV.

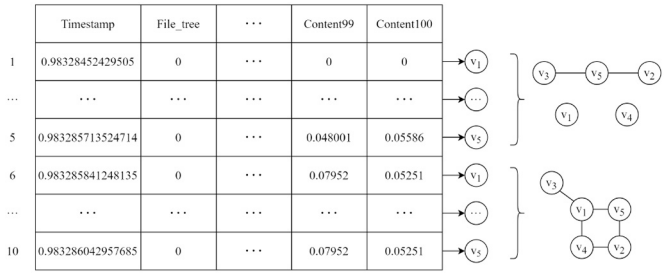


Fig. 3. The process of converting activity logs into graphs.

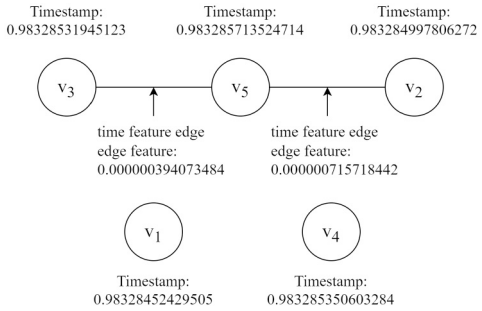


Fig. 4. The graph composed of the first 5 records of ACM2278 activity log through definition 1.

C. Graph Structure

To capture the contextual relationship of different behaviors over a period of time and give full play to the advantages of the Word2Vec, the user activity log is transformed into a graph with topological structure.

The graph construction method is based on the theory proposed by Li et al. [42]. They proposed two methods for converting time series data into graphs, one is Visibility Graph (VG), and the other is Horizontal Visibility Graph (HVG), which is defined as follows.

Definition 1 (VG): Given a time series $T = (t_1, \ldots, t_n)$, its VG representation $G = (V, E)$ has n nodes, where $V = (v_1, \ldots, v_n)$. An edge $e = (i, j) \in E$ iff $\forall k$, $i < k < j$, $(1 \leq i, j \leq n)$, such that inequality $t_k < t_j + (t_i - t_j)\frac{j-k}{j-i}$ is satisfied.

Definition 2 (HVG): Given a time series $T = (t_1, \ldots, t_n)$, its HVG representation $\hat{G} = (V, E)$ has n nodes, $V = (v_1, \ldots, v_n)$. An edge $e = (i, j) \in E$ iff $\forall k$, $i < k < j$, $(1 \leq i, j \leq n)$, such that inequality $t_i, t_j > t_k$ is satisfied.

For insider activity log processing, in the above definitions, T denotes the user activity log, and $t_i, (i = 1, 2, \ldots, n)$ is the energy value of i-th activity item. Mean value calculated according to the 113 fields of each log record (e.g., log records of ACM2278 in Table IV), is used as the related energy value. It is proved in [42] that the VG can represent the global features of time series, while the HVG can represent the local features.

To construct the graph structure of user activity log, each activity item is considered as a node, and each graph contains n nodes. Let $V \in \mathbb{R}^n$ be a vector as

$$V = \{v_1, v_2, \ldots, v_n\} \tag{4}$$

where v_i is the i-th node. The left part of Fig. 3 shows the process of converting activity logs into nodes when $n = 5$.

Let vector $E \in \mathbb{R}^z$ consist of the undirected edges of the graph structure to represent the contextual relationship between nodes,

$$E = \{e_1, e_2, \ldots, e_z\} \tag{5}$$

where e_i is the i-th undirected edge in the graph. For example, $e_1 = (v_1, v_2)$ represents that there is an undirected edge between nodes v_1 and v_2. The right part of Fig. 3 shows the graph constructed by definition 1 when $n = 5$ in Eq. (4). The connection method and number of edges are determined according to the definition (1 or 2) used and the energy value of activity item.

Let G represent a graph composed of nodes and undirected edges, which can be constructed with definition 1 or 2 and denoted as

$$G = (V, E) \tag{6}$$

The nodes represent the position of activity logs in the graph, while the undirected edges represent the contextual relationship between activity logs.

In Table IV, except for the "timestamp" field, the values of the other fields are stored in the corresponding nodes as node features. The value of node features represented by a vector $B \in \mathbb{R}^{112}$ is written as

$$B = \{b_1, b_2, \ldots, b_{112}\} \tag{7}$$

where b_i is the i-th numerical representation in Table IV (calculated from "File_tree" field). For example, b_1 is the numerical representation of the "File_tree" field, b_2 is the numerical representation of the "Activity" field, and b_{112} is the numerical representation of the 100-th word in the "Content" field.

Let vector $\hat{V}$ represent the set of nodes in the graph,

$$\hat{V} = \{\hat{v}_1, \hat{v}_2, \ldots, \hat{v}_n\} \tag{8}$$

where $\hat{v}_i$ is the i-th node which stores the corresponding node feature B_i.

The edges in the graph can also store features. According to the stored edge features, the edges can be divided into three types: Time-based edge, distance-based edge and attention-based edge.

The time-based edge stores the absolute value of the difference of the timestamps corresponding to the two nodes at the ends of this edge. This means that the time-based edge feature represents the time interval between the two activity logs. Fig. 4 shows the graph composed of the first five records of ACM2278 activity log based on definition 1, and the feature calculation display of time feature edges.

Euclidean distance is used to calculate the distance-based edge feature, written as

$$dist(\hat{v}_i, \hat{v}_j) = \sqrt{\sum_{k=1}^{112} (\hat{v}_{ik} - \hat{v}_{jk})^2} \tag{9}$$

where $\hat{v}_i$ and $\hat{v}_j$ are the nodes at both ends of an edge. $\hat{v}_{ik}$ represents the k-th value (b_k) of the node feature (B_i) stored

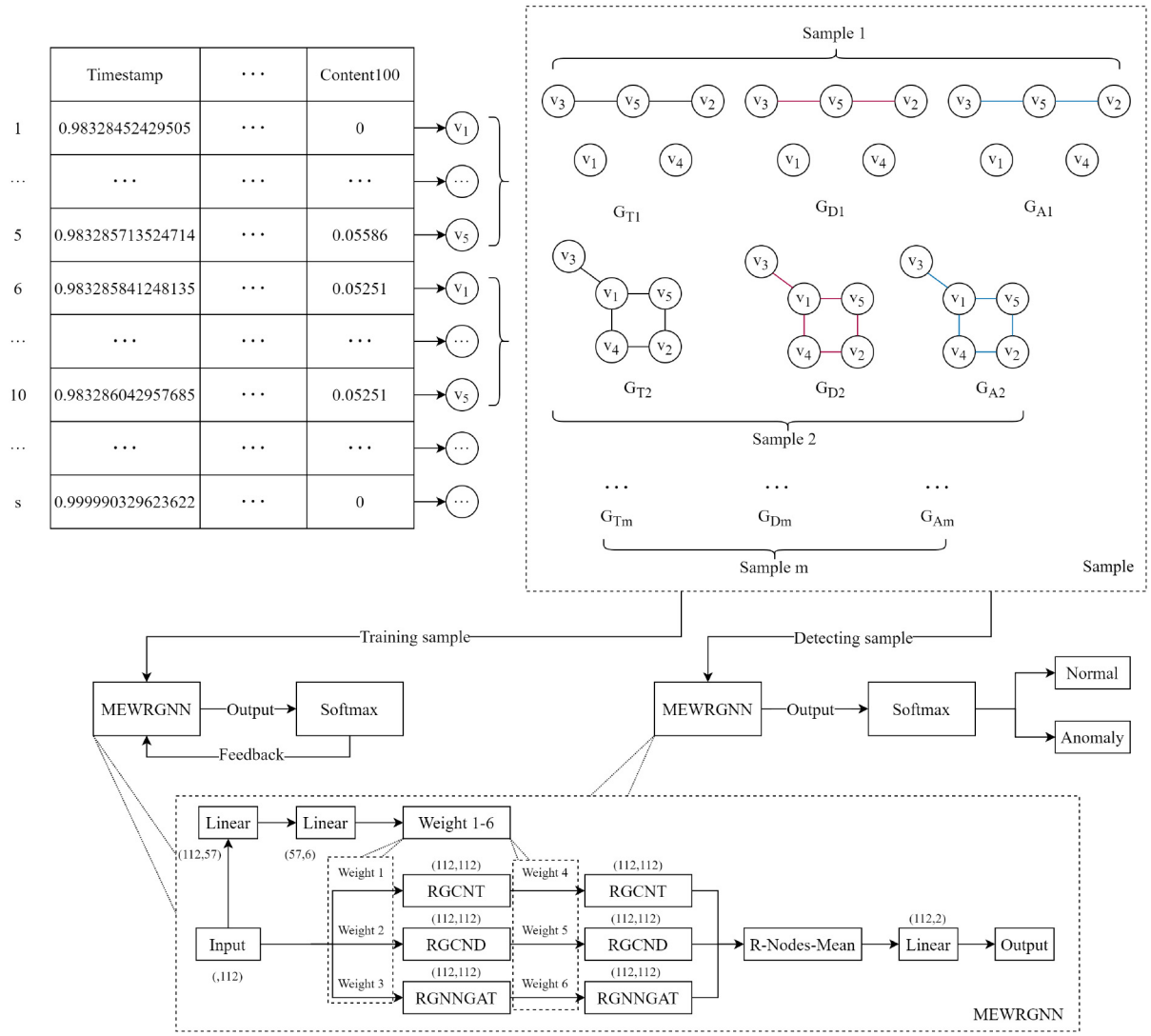


Fig. 5. The graph construction of the activity log and the training and detection process of the MEWRGNN model.

in the *i*-th node (v_i). $\hat{v}_{jk}$ is similar to $\hat{v}_{ik}$. The distance-based edge feature indicates the similarity of the two corresponding activity logs. Euclidean distance is chosen as the distance-based edge feature since it is more suitable for calculating two rows of data with numerical values, such as floating-point and integer. It was applied in most instance-based learning models [41].

The attention-based edge feature is automatically calculated by the MEWRGNN model, which will be described in detail later.

Let vectors $\hat{E}_T, \hat{E}_D$ and $\hat{E}_A \in \mathbb{R}^z$ represent the sets of edge features of time feature edges (i.e., time-based edges), distance feature edges (i.e., distance-based edges) and attention feature edges (i.e., attention-based edges), respectively, written as

$$\hat{E}_T = \{\hat{e}_{T1}, \hat{e}_{T2}, \ldots, \hat{e}_{Tz}\} \tag{10}$$

$$\hat{E}_D = \{\hat{e}_{D1}, \hat{e}_{D2}, \ldots, \hat{e}_{Dz}\} \tag{11}$$

$$\hat{E}_A = \{\hat{e}_{A1}, \hat{e}_{A2}, \ldots, \hat{e}_{Az}\} \tag{12}$$

where $\hat{e}_{Ti}$, $\hat{e}_{Di}$ and $\hat{e}_{Ai}$ are the edge features of the *i*-th time feature edge, distance feature edge and attention feature edge, respectively, which store the corresponding edge feature values.

Let $\hat{G}_T$, $\hat{G}_D$ and $\hat{G}_A$ represent the graphs using time-based edges, distance-based edges and attention-based edges, respectively, and they are written as

$$\hat{G}_T = \left(\hat{V}, \hat{E}_T\right) \tag{13}$$

$$\hat{G}_D = \left(\hat{V}, \hat{E}_D\right) \tag{14}$$

$$\hat{G}_A = \left(\hat{V}, \hat{E}_A\right) \tag{15}$$

The input sample for the proposed model is composed of three types of graphs, i.e., $\hat{G}_T$, $\hat{G}_D$ and $\hat{G}_A$ corresponding to the types of edge features $\hat{E}_T$, $\hat{E}_D$ and $\hat{E}_A$, respectively. As shown in the upper part of Fig. 5 (where G_T, G_D and G_A represent the graphs using time-based edges, distance-based edges and attention-based edges, respectively.), if the graph is constructed with 5 nodes (in Eq. (8), $n = 5$). These 5 nodes can be used to construct 3 graphs ($\hat{G}_T$, $\hat{G}_D$ and $\hat{G}_A$ which are corresponding to G_T, G_D and G_A, separately) with the same topology and node features but different edge features.

The constructed 3 graphs are treated as an input sample of the MEWRGNN model.

The constructed graph will be marked as an anomaly graph if there are nodes with malicious activity logs in this graph. Otherwise, it will be labeled as normal graph if there are no nodes with malicious activity logs in the graph.

D. Graph Convolutional Network

GCN [43] is a classic GNN originally used for semi-supervised classification of graph structure data. Its multi-layer case can be formed by the layer-wise rule as

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right) \tag{16}$$

where $\tilde{A} = A + I$ indicates the adjacency matrix of the graph which is added self-connections, A is the adjacency matrix of the undirected graph, $I \in \mathbb{R}^{N\times N}$ is the identity matrix, $H^{(l)} \in \mathbb{R}^{N\times D}$ is the matrix of activations in the l-th layer, $W^{(l)}$ represents the shared trainable weight matrix for node-wise feature transformation, $\sigma(\cdot)$ is the activation function.

E. Relational Graph Convolutional Network

R-GCN [44] is an extension of the classic model GCN. GCN only explores the topological structure features of data to obtain node representation but does not fully explore the edge information on the graph. While the R-GCN supplements this aspect, and it can be formulated as

$$h_i^{(l+1)} = \sigma\left(W_0^{(l)}h_i^{(l)} + \sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}_i^r}\frac{1}{c_{i,r}}W_r^{(l)}h_j^{(l)}\right) \tag{17}$$

where $h_i^{(l)} \in \mathbb{R}^{d^{(l)}}$ is the hidden state of node v_i in the l-th layer of the neural network, $d^{(l)}$ is the dimension of this layer's representation, $\mathcal{N}_i^r$ denotes the set of one-hop neighbors of node v_i under relation $r \in \mathcal{R}$. $c_{i,r} = |\mathcal{N}_i^r|$ denotes a normalization constant, $|\mathcal{N}_i^r|$ is the number of one-hop neighbors of node v_i under relation r.

F. CAN-Graph Attention Networks

CAN-GAT [45] is a variant of Graph Attention Networks (GAT) [46], and it can be formulated as

$$z_i^{(l)} = ReLU\left(W^{(l)}h_i^{(l)}\right) \tag{18}$$

$$e_{ij}^{(l)} = LeakyReLU\left(\left(\vec{V}_1^{(l)^T}z_i^{(l)}\right)\cdot\left(\vec{V}_2^{(l)^T}z_j^{(l)}\right)\right) \tag{19}$$

$$\alpha_{ij}^{(l)} = \frac{\exp\left(e_{ij}^{(l)}\right)}{\sum_{k\in\mathcal{N}(i)}\exp\left(e_{ik}^{(l)}\right)} \tag{20}$$

$$h_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)}\alpha_{ij}^{(l)}z_j^{(l)}\right) \tag{21}$$

where $h_i^{(l)}$ is the node features of the i-th node of l-th layer, $W^{(l)}$ is the learnable weight matrix of l-th layer. $ReLU(\cdot) = max(0,\cdot)$ is non-linear activation function. $\vec{V}_1^{(*)^T}$ and $\vec{V}_2^{(*)^T}$ are the learnable weight vectors. $\mathcal{N}(i)$ denotes the set of one-hop neighbors of node v_i. "$\cdot$" denotes dot-product operation. $(\cdot)^T$ represents transpose, and $LeakyReLU(\cdot)$ is a non-linear activation function, written as

$$y_i = \begin{cases} x_i & x_i \geq 0 \\ \frac{x_i}{b_i} & x_i < 0 \end{cases} \tag{22}$$

where $b_i \in (1, +\infty)$, $b_i = 100$ is set as the default value.

IV. Anomaly Detection Using MEWRGNN Model

A. Approach Overview

In insider threat detection, GNN method is a powerful tool for graph structure data processing to facilitate extracting the contextual relationship between different behaviors of users over a period of time. It can achieve excellent performance in the classification of various graph structures [47], [48] and has significant potential for insider threat detection [17]. For processing the text information in activity log efficiently with GNN-like method, Word2Vec is used to derive the numeric representations of the text. In this paper, the MEWRGNN model is designed to capture the contextual relationship between different behaviors of users over a period of time. The designed model is shown in Fig. 5 where the upper part is the process of transforming the activity log into a graph, the lower left part is the training process, and the lower right part presents the detection process. The constructed graph transformation process is applied to explore the contextual relationship between activity logs over time to form input sample for the MEWRGNN model. The training process captures the potential features of the activity log, and obtains a suitable trained model, while the detection process determines whether the activity logs are abnormal or not via using the trained model.

B. MEWRGNN

The proposed MEWRGNN model is implemented based on three models presented bellow (RGCNT, RGCND and RGNNGAT). The input samples for the MEWRGNN Model are three kinds of graphs ($\hat{G}_T$, $\hat{G}_D$ and $\hat{G}_A$). First, the node feature in $\hat{V}$ of the input sample is linearly transformed twice, and then 6 weights are obtained through the softmax function, written as

$$y_1 = W_1\hat{V} + b_1 \tag{23}$$

$$y_2 = W_2 y_1 + b_2 \tag{24}$$

$$M = softmax(y_2) \tag{25}$$

where W_i and b_i (i = 1,2) are the learnable parameter matrix and bias, respectively. Eq. (23) and Eq. (24) are related to the calculation of the fully connected layers of the classic neural network. The dimension of input sample is reduced from 112 to 57 through the first layer (Eq. (23)), and to 6 further through the second layer (Eq. (24)). *softmax* is regarded as a normalized function. Let $M \in \mathbb{R}^6$ be a vector, written as

$$M_i = \frac{exp(y_{2i})}{\sum_j^6 exp(y_{2j})} \tag{26}$$

where y_{2i} and y_{2j} are the values of i-th and j-th dimensions of y_2, respectively. M_i is the value of i-th dimension to be

normalized. Eqs. (23)-(25) are the calculation of the learning parameters from node features to weights (M_1, M_2, M_3, M_4, M_5, M_6), of which the process is shown in the lower part of Fig. 5.

Input samples $\hat{G}_T$, $\hat{G}_D$ and $\hat{G}_A$ are used for feature extraction with different GNN layers.

Relational Graph Convolutional Network Time (RGCNT) layers process the input sample $\hat{G}_T$ to capture time features of activity log. The related calculations are written as

$$h_{T_i}^{(2)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \frac{1}{c_{ij}} W_T^{(1)}\left(h_{T_j}^{(1)} \hat{e}_{T_{ij}} M_1\right)\right) \tag{27}$$

$$h_{T_i}^{(3)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \frac{1}{c_{ij}} W_T^{(2)}\left(h_{T_j}^{(2)} \hat{e}_{T_{ij}} M_4\right)\right) \tag{28}$$

where $W_T^{(1)}$ is the shared trainable weight matrix of first layer, $h_{T_j}^{(1)}$ denotes the feature of the j-th node in $\hat{G}_T$, $\hat{e}_{T_{ij}}$ represents edge feature of the edge connecting nodes $\hat{v}_i$ and $\hat{v}_j$, $h_{T_i}^{(2)}$ and $W_T^{(2)}$ are respectively used as the input and the shared trainable weight matrix of the second layer of RGCNT, the meaning of other symbols are similar to what they are in Eq. (17).

Relational Graph Convolutional Network Distance (RGCND) layers process the input sample $\hat{G}_D$ to capture the distance features (similarity) of activity log. The related calculations are written as

$$h_{D_i}^{(2)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \frac{1}{c_{ij}} W_D^{(1)}\left(h_{D_j}^{(1)} \hat{e}_{D_{ij}} M_2\right)\right) \tag{29}$$

$$h_{D_i}^{(3)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \frac{1}{c_{ij}} W_D^{(2)}\left(h_{D_j}^{(2)} \hat{e}_{D_{ij}} M_5\right)\right) \tag{30}$$

where $W_D^{(1)}$ is the shared trainable weight matrix of first layer, $h_{D_j}^{(1)}$ denotes the feature of the j-th node in $\hat{G}_D$, $\hat{e}_{D_{ij}}$ represents edge feature of the edge connecting nodes $\hat{v}_i$ and $\hat{v}_j$, $h_{D_i}^{(2)}$ and $W_D^{(2)}$ are respectively used as the input and the shared trainable weight matrix of the second layer of RGCND, the meaning of other symbols are similar to what they are in Eq. (17).

Relational Graph Neural Network Graph Attention Networks (RGNNGAT) layers process the input sample $\hat{G}_A$ to capture importance features of activity log. The RGNNGAT of the first layer is written as:

$$z_i^{(1)} = ReLU\left(W_A^{(1)} h_{A_i}^{(1)}\right) \tag{31}$$

$$e_{ij}^{(1)} = LeakyReLU\left(\left(\vec{V}_1^{(1)^T} z_i^{(1)}\right) \cdot \left(\vec{V}_2^{(1)^T} z_j^{(1)}\right)\right) \tag{32}$$

$$\alpha_{ij}^{(1)} = \frac{\exp\left(e_{ij}^{(1)}\right)}{\sum_{k\in\mathcal{N}(i)} \exp\left(e_{ik}^{(1)}\right)} M_3 \tag{33}$$

$$h_{A_i}^{(2)} = \sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(1)} z_j^{(1)} \tag{34}$$

where the $\alpha_{ij}^{(1)}$ calculated by Eq. (33) is stored in $\hat{e}_{A_{ij}}$ as an edge feature. Eq. (34) update node features $h_{A_i}^{(2)}$ according to the value of edge feature $\alpha_{ij}^{(1)}$. In the same way, the second layer of RGNNGAT is written as:

$$z_i^{(2)} = ReLU\left(W_A^{(2)} h_{A_i}^{(2)}\right) \tag{35}$$

$$e_{ij}^{(2)} = LeakyReLU\left(\left(\vec{V}_1^{(2)^T} z_i^{(2)}\right) \cdot \left(\vec{V}_2^{(2)^T} z_j^{(2)}\right)\right) \tag{36}$$

$$\alpha_{ij}^{(2)} = \frac{\exp\left(e_{ij}^{(2)}\right)}{\sum_{k\in\mathcal{N}(i)} \exp\left(e_{ik}^{(2)}\right)} M_6 \tag{37}$$

$$h_{A_i}^{(3)} = \sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(2)} z_j^{(2)} \tag{38}$$

The node features calculated from each GNN layer need to be fused to facilitate classification. The node features of each graph are fused into h_g as

$$h_g = \sum_{r\in\mathcal{R}} \frac{\sum_{j=1}^{N} h_{r_j}^{(3)}}{N} \tag{39}$$

where $\mathcal{R} = \{T, D, A\}$. To implement anomaly detection, the dimension of the fusion features h_g is reduced from 112 to 2 with the formula as

$$y_3 = W_3 h_g + b_3 \tag{40}$$

where $y_3 \in \mathbb{R}^2$ denotes the output of a linear transformation calculation, and it is fed into the *softmax* function to achieve predicted results,

$$Y'_{P_i} = \frac{\exp(y_{3_i})}{\sum_{j=1}^{k} \exp\left(y_{3_j}\right)} \tag{41}$$

where $Y_{P_i}{}', \forall i = 1, 2$, represents the probability that the prediction result is normal or abnormal, and $k = 2$ is the number of categories. Eqs. (23)-(41) constitute a complete MEWRGNN model in which the GNN layer structure is shown in the lower part of Fig. 5.

For the train calculation of the MEWRGNN model, the loss function to evaluate the error between predicted category and true category is defined as

$$L_{eer}(w, b) = -\frac{1}{m} \sum_{i=1}^{m} Y_i \log\left(Y'_{P_i}(w, b)\right) \tag{42}$$

where Y_i denotes the true category and Y'_{P_i} denotes predicted one, w and b are the weight and bias of the MEWRGNN model in hidden layers respectively.

As batches of groups of inputs are applied to train the MEWRGNN model, the actual summation effect is the loss of a batch L_{bat}, written as

$$L_{bat}(w, b, u) = \sum_{i=1}^{u} L_{eer}^{(i)}(w, b) \tag{43}$$

where u denotes the number of groups of inputs to form a batch. In this paper, $u = 5$ denotes that a batch is composed of 5 graph structure data of activity logs.

TABLE V
THE NUMBER OF ACTIVITY LOGS OF NORMAL USERS

Normal user	DNS1758	RSR2681	QCD0386	RJH0219
Normal activity	55,898	39,132	50,970	42,191

The $L_{bat}(w, b, u)$ is selected as the final loss function and optimization object in the training phase of the MEWRGNN model. The optimal target is to minimize the error calculated by the loss function. In the training phase, the Adam optimization algorithm [49] is used to search parameter set P^*,

$$P^* = \arg \min_{\{w,b\} \in \Omega} L_{bat}(w, b, u) \tag{44}$$

By using parameter set P^*, the MEWRGNN model can implement insider threat detection for user activity logs.

V. EVALUATION

In this section, the implementation of the presented MEWRGNN model, and the experimental settings are explained first. Then the effectiveness, accuracy and interpretability of the MEWRGNN model are evaluated. Comparison experiments with several baselines are presented. Following by the discussion of the efficiency evaluation.

A. MEWRGNN Model Implementation

The architecture of the proposed MEWRGNN Model is shown in Fig. 5. One of its input samples includes three graphs with different edge features ($\hat{G}_T$, $\hat{G}_D$ and $\hat{G}_A$). First, the node features stored in $\hat{V}$ undergo several linear transformations to obtain the weight feature M_i $(i = 1, 2, \ldots, 6)$ (with Eqs. (23)-(26)). Then, the input samples with different edge features are processed by different GNN layers to obtain salient features (with Eqs. (27)-(38)). Finally, these salient features undergo fusion and feature transformation to output detection results (with Eqs. (39)-(41)). In the training phase, the number of training iterations of each model is set to 5, and the Adam method with a learning rate of 0.001 is used for model training.

B. User Activity Log Dataset

The CERT Insider Threat Dataset version r6.2 is applied as original experimental data. The activity logs of malicious users (ACM2278, CMP2946, PLJ1771 and CDE1846) are extracted from the dataset in chronological order to facilitate analysis. Activity logs data of MBG3183 are used as test data only, since there are just four abnormal activity logs in the dataset which is insufficient to generate enough training and test data. And the activity logs of 500 normal users (such as DNS1758, RSR2681, QCD0386 and RJH0219) in the dataset are selected for comparison experiments. The number of activity logs of malicious users and four normal users is shown in Table I and Table V, respectively.

C. Accuracy Evaluation

The accuracy, recall, precision, F1 scores, and False Positive Rate (FPR) are calculated by collecting the number of true negatives (TN), true positives (TP), false negatives (FN), and false positives (FP).

TABLE VI
DETECTION ACCURACY OF MEWRGNN MODEL WITH DIFFERENT CONFIGURATIONS,(ACCURACY RECORDED IN PERCENTAGE %)

Method	VG	HVG	VG-HVG	VG	HVG	VG-HVG
$n = 20$		P_{cert} = 60%			P_{cert} = 80%	
ACM2278	98.26204	98.00000	98.12220	98.00705	97.77411	98.09411
CMP2946	87.08228	87.11934	87.10748	87.39169	87.60385	87.29376
PLJ1771	96.38896	96.28156	96.15820	96.94186	97.35465	97.47383
CDE1846	85.15532	86.55895	86.67913	88.48979	87.42630	88.06802
Average	91.72215	91.98997	92.01675	92.70760	92.53973	92.73243
$n = 30$		P_{cert} = 60%			P_{cert} = 80%	
ACM2278	98.53439	98.28975	98.32155	98.58657	97.51764	98.20000
CMP2946	87.22914	87.22803	86.70189	87.97327	85.95545	87.34075
PLJ1771	97.00871	96.18300	95.80610	99.56332	97.62008	97.83842
CDE1846	87.60374	86.43537	87.51190	89.79592	88.50000	88.54421
Average	92.59400	92.03404	92.08536	**93.97977**	92.39829	92.98085
$n = 50$		P_{cert} = 60%			P_{cert} = 80%	
ACM2278	97.72058	97.51764	98.20000	97.28654	97.30994	96.79532
CMP2946	86.76066	86.98886	86.86270	87.05185	86.48518	86.94814
PLJ1771	97.15636	97.80000	97.38545	97.14388	97.65467	97.44604
CDE1846	86.89235	87.72237	86.97733	87.72471	87.41573	88.19662
Average	92.13249	92.50722	92.35637	92.30175	92.21638	92.34653
$n = 70$		P_{cert} = 60%			P_{cert} = 80%	
ACM2278	97.70370	98.49382	98.11111	97.14754	96.96721	97.24590
CMP2946	87.01302	86.67447	87.12500	87.53886	87.07772	87.34715
PLJ1771	97.49489	98.09693	97.77551	97.11111	96.81818	96.54545
CDE1846	88.19521	87.69721	88.23505	87.80952	87.61111	87.76984
Average	92.60171	92.74061	92.81167	92.40176	92.11856	92.22709

$$precision = \frac{TP}{TP + FP}, \tag{45}$$

$$accuracy = \frac{TP + TN}{TP + FP + FN + TN}, \tag{46}$$

$$recall = \frac{TP}{TP + FN}, \tag{47}$$

$$F1 = \frac{precision \times recall}{precision + recall} \times 2, \tag{48}$$

$$FPR = \frac{FP}{FP + TN}. \tag{49}$$

Table VI shows the detection accuracy results of the MEWRGNN model with different configurations, and the accuracy is recorded in percentage. For the detection of each user, the MEWRGNN model is trained with P_{cert} of the user's activity logs, and then is applied to detect the rest $(1 - P_{cert})$ of user's activity logs. Such an experiment is implemented 100 times with the same experimental setup for the user detection, and the mean value of the obtained 100 detection accuracy values for the related user is listed in the table. VG and HVG denote that the input sample for the model is a graph constructed with Definition 1 and Definition 2, respectively. VG-HVG represents that the first input sample of the model is the graph constructed according to Definition 1, and the second input sample graph is constructed with Definition 2, the other input samples are constructed by these two methods alternately. $n = 20$ denotes that the value of n in Eq. (4), i.e., a graph consists of 20 nodes (activity logs). P_{cert} (e.g., 60%) represents the percentage (e.g., 60%) of each user's normal and abnormal activity logs that are used as the training datasets, while the rest are used as the test datasets. For constructing abnormal datasets, the abnormal activity logs of each abnormal user are put together to form the abnormal datasets. P_{cert} (e.g., 60%) of

TABLE VII
DETECTION RESULTS OF MEWRGNN IN THE CASES OF USING DIFFERENT NUMBER OF WORDS SELECTED AS FEATURES FROM THE "CONTENT" FIELD, (ACCURACY, PRECISION, RECALL AND F1 SCORE ARE RECORDED IN PERCENTAGE %)

n_{words} = 50	Accuracy	Precision	Recall	F1 score
ACM2278	98.58657	98.58956	98.58657	98.58238
CMP2946	87.75056	89.38184	87.75056	85.53503
PLJ1771	99.56332	99.56646	99.56332	99.56290
CDE1846	89.79592	91.10975	89.79592	89.13999
Average	93.92409	94.66190	93.92409	93.20508
n_{words} = 100	Accuracy	Precision	Recall	F1 score
ACM2278	98.58657	98.58956	98.58657	98.58238
CMP2946	87.97327	89.54959	87.97327	85.85810
PLJ1771	99.56332	99.56646	99.56332	99.56290
CDE1846	89.79592	91.10975	89.79592	89.13999
Average	**93.97977**	**94.70384**	**93.97977**	**93.28584**
n_{words} = 150	Accuracy	Precision	Recall	F1 score
ACM2278	98.58657	98.58956	98.58657	98.58238
CMP2946	87.97327	89.54959	87.97327	85.85810
PLJ1771	99.12664	99.12664	99.12664	99.12664
CDE1846	89.79592	91.10975	89.79592	89.13999
Average	93.87060	94.59389	93.87060	93.17678
n_{words} = 200	Accuracy	Precision	Recall	F1 score
ACM2278	98.58657	98.58956	98.58657	98.58238
CMP2946	87.97327	89.17341	87.97327	85.97508
PLJ1771	99.12664	99.12664	99.12664	99.12664
CDE1846	89.45578	90.57220	89.45578	88.80797
Average	93.78557	94.36545	93.78557	93.12302

TABLE VIII
DETECTION RESULTS FOR USER ACM2278 WHEN THE MODEL USES DIFFERENT AGGREGATION FUNCTIONS, (ACCURACY, PRECISION, RECALL AND F1 SCORE ARE RECORDED IN PERCENTAGE %)

Aggregation function	Accuracy	Precision	Recall	F1 score
Arithmetic average	98.58657	98.58956	98.58657	98.58238
Median	98.58657	98.58956	98.58657	98.58238
Mode	98.23322	98.24291	98.23322	98.22526

TABLE IX
THE ACCURACY RESULTS OF THE MEWRGNN MODEL ON THE DETECTION OF NORMAL USER ACTIVITY LOGS

User	DNS1758	RSR2681	QCD0386	RJH0219
Accuracy (%)	97.91379	98.36079	98.97679	96.81989

TABLE X
DETECTION ACCURACY OF USERS' NORMAL ACTIVITY PRIOR TO LEAVING THE COMPANY, AFTER THEY ARE DETECTED AS ANOMALOUS USERS

User	ACM2278	CMP2946	PLJ1771	CDE1846
Accuracy (%)	98.59154	100.00000	100.00000	98.25000

TABLE XI
DETECTION RESULTS OF THE CASES THAT MODEL TRAINING AND TESTING USE DIFFERENT USER DATA (DATA IN "TEST" COLUMN ARE ALL FROM THE NORMAL ACTIVITY LOGS OF THE CORRESPONDING USER, AND THE ACCURACY, PRECISION, RECALL AND F1 SCORE ARE RECORDED IN PERCENTAGE %)

Train	Test	Precision	Accuracy	Recall	F1
PLJ1771	DNS1758	100.00000	99.73190	99.73190	99.86577
DNS1758	PLJ1771	100.00000	97.82609	97.82609	98.90110
DNS1758	RJH0219	100.00000	100.00000	100.00000	100.00000
PLJ1771	CDE1846	100.00000	100.00000	100.00000	100.00000

TABLE XII
DETECTION RESULTS OF THE CASES THAT TRAINING AND TESTING PHASE USE DIFFERENT USER DATA ("TEST" COLUMN DATA ARE ALL FROM THE ABNORMAL ACTIVITY LOG OF THE CORRESPONDING USER, AND THE ACCURACY, PRECISION, RECALL AND F1 SCORE ARE RECORDED IN PERCENTAGE %)

Train	Test	Precision	Accuracy	Recall	F1
PLJ1771	CDE1846	100.00000	100.00000	100.00000	100.00000
DNS1758	CDE1846	100.00000	100.00000	100.00000	100.00000

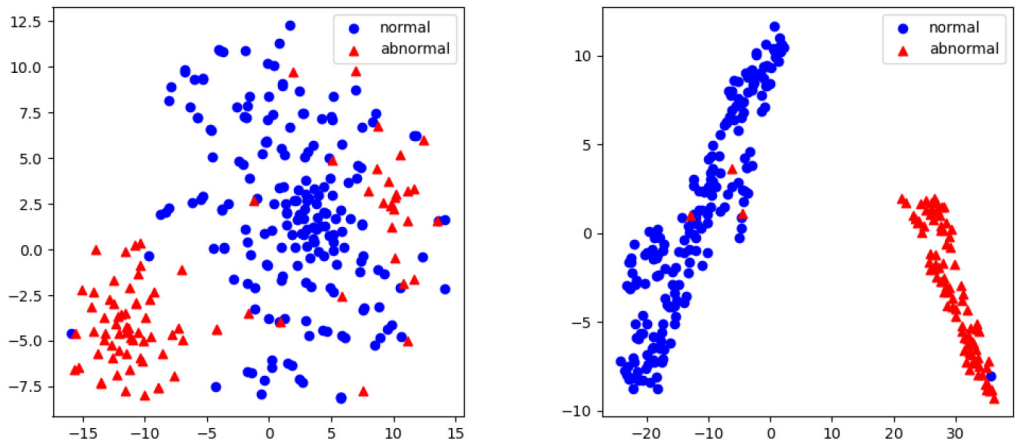
the abnormal activity logs of each abnormal user constitute the abnormal training datasets, and the rest constitute the abnormal test datasets. It can be seen from Table VI that the MEWRGNN model has good robustness, the performance of the model is not affected greatly in the cases that the graph construction method, the number of nodes used in one constructed graph, and the percentage of training datasets, vary in a certain range. An arithmetic average value of detection accuracy results is calculated for each parameter configuration model, to evaluate the model's overall performance (As is the "Average" in the table). The table shows that, by using the definition 1 method to construct the graph of 30 nodes as an input, 80% of the activity logs as the training datasets, the model obtains the best results. Next, we choose this group of configuration parameters for the model to conduct an in-depth study.

Table VII shows the experimental results of sensitivity analysis on the number of words (indicated with n_{words}) selected as features in the "Content" field, with the parameter configuration selected from Table VI. We can see that the model obtains the best performance by selecting the first 100 words in the "content" field as features.

In general, insider threat detection model should has good capability to detect abnormal user activities and recognize normal user activities. Table IX shows the detection accuracy performance of the proposed model on normal user activity logs. It indicates that the proposed method has a suitable recognition rate for normal user activity logs.

The activity log dataset of abnormal user often contains a large number of normal activity logs and limited abnormal activity logs, since an abnormal user usually performs normal activities after implementing threat activities that will generate corresponding logs. Table X shows the recognition accuracy of these abnormal users' normal activity logs. It shows even if the users have been identified as abnormal, their normal activity logs can still be classified correctly.

To verify that the proposed model can capture the behavior characteristics of each user well, we select the training datasets from one of the users and the test datasets from the different ones to conduct simulation experiments. Table XI and Table XII are the detection results of the cases that the training and testing data do not come from the same user simultaneously. The column"train" in the table represents the model is trained with the corresponding user training datasets. E.g.,



(a) The visualization of the ACM2278 test datasets which are not processed by the MEWRGNN model.

(b) The visualization of the ACM2278 test datasets which are processed by the MEWRGNN model.

Fig. 6. The visualization of the user ACM2278's test datasets using t-SNE technology.

the user PLJ1771 in the "train" column in the first row of Table XI means that the user PLJ1771's training datasets are used to train the model. The user data in the "test" column in Table XI all come from the normal activity logs in the respective test datasets. While the user data in the "test" column in Table XII all come from the abnormal activity logs in the respective test datasets. It can be seen from Table XI that, if someone logs into the system with another person's account, the model will treat him as an abnormal user even if his operation is normal. Table XII shows that the user using other person's accounts to enter the system and perform malicious operations are more likely to be detected.

The above experiment results demonstrate that the proposed model can be applied to insider threat detection in various scenarios, and the model has strong stability.

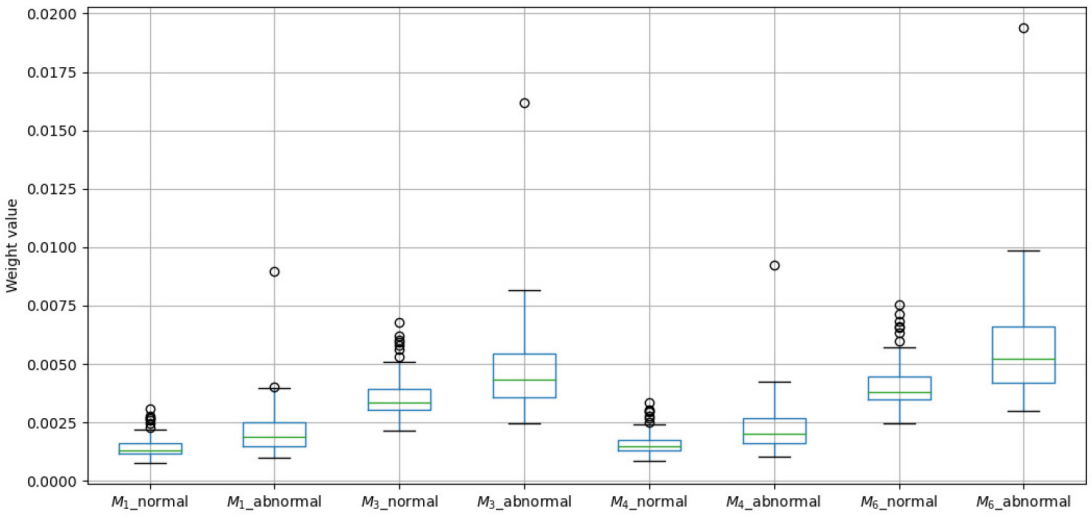
TABLE XIII
THE INFLUENCE OF EACH WEIGHT ON THE DETECTION OF THE MODEL TRAINED WITH THE DATA OF USERS PLJ1771 AND QCD0386, RESPECTIVELY, (ACCURACY RECORDED IN PERCENTAGE %)

PLJ1771				Accuracy	99.56331
M_1	M_2	M_3	M_4	M_5	M_6
0.0075	0.4637	0.0081	0.0077	0.5057	0.0074
-D1D2	-A1A2	-T1T2	-T1T2A1A2	-T1T2D1D2	-D1D2A1A2
60.26200	99.56331	99.56331	98.68995	60.26200	60.26200
-D2	-A2	-T2	-T2A2	-T2D2	-A2D2
60.26200	99.56331	99.56331	99.56331	51.52838	60.26200
QCD0386				Accuracy	99.76798
M_1	M_2	M_3	M_4	M_5	M_6
0.0064	0.5791	0.0065	0.0051	0.3967	0.0063
-D1D2	-A1A2	-T1T2	-T1T2A1A2	-T1T2D1D2	-D1D2A1A2
78.88631	99.76798	99.76798	99.30394	78.88631	78.88631
-D2	-A2	-T2	-T2A2	-T2D2	-A2D2
78.88631	99.76798	99.76798	99.76798	78.88631	78.88631

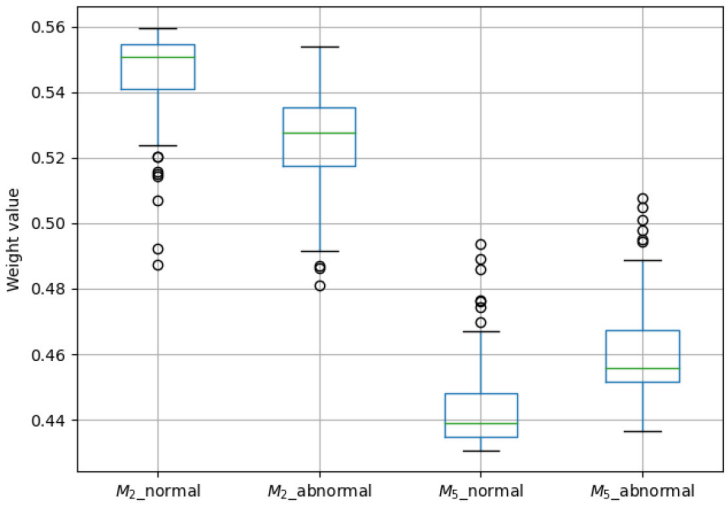
D. Interpretability Evaluation

The data features extracted by the MEWRGNN model can be visualized. Fig. 6 shows the scenario of the user ACM2278's test datasets while using the t-SNE [50] technology for dimensionality reduction and visualization. Fig. 6 (a), shows the scene that the datasets are directly visualized by using the t-SNE technology. It can be seen that normal data points and abnormal data points are mixed and closely spaced. The two types of data are inseparable linearly. The visualization result of the ACM2278 test datasets processed by the MEWRGNN model are displayed in Fig. 6 (b), which shows that the normal and abnormal data are separated clearly, and are almost linearly separable. Fig. 6 shows that the MEWRGNN model is good at extracting abstract feature representation and feature transformation.

To analyze the MEWRGNN model from a quantitative perspective, we focus on explaining the 6 weights $(M_i,\ i = 1, \ldots, 6)$ of the model. Each weight is closely related to the update of node features and edge features of a certain layer. For example, the weight M_1 is related to the update of the RGCNT node of the first layer and the time feature edge (Eq. (27)). We separately analyze the MEWRGNN model trained with the activity logs of users PLJ1771 and QCD0386 to explain the impact of different types of features on the model's performance. Table XIII shows the influence of each weight on the detection of the model trained with the data of users PLJ1771 and QCD0386, respectively. In the table, accuracy represents the detection accuracy of the entire MEWRGNN model. The number listed below $(M_i,\ i = 1, \ldots, 6)$ is the value of the weight. $-\mathcal{R}g$ $(\mathcal{R} = \{T, D, A\},\ g = \{1, 2\})$ denotes to delete the corresponding GNN layer on the trained MEWRGNN model to test the effect of this layer on the detection ability. For example, -D1D2 means to test the accuracy of the model after deleting Eqs. (29)-(30) from the trained MEWRGNN model, and -T2A2 means to test the accuracy of the model after deleting Eqs. (28), (35)-(38) from the trained MEWRGNN model. It can be seen from Table XIII that M_2 and M_5 have the largest weight proportions, and they are related to the RGCND layer. The update of the node feature in the RGCND layer depends on distance feature edges. If the RGCND layer of the model is deleted (-D1D2, -T1T2D1D2, -D1D2A1A2, -D2, -T2D2, -A2D2), the detection accuracy of the model directly



(a) The distribution of M_1, M_3, M_4 and M_6.



(b) The distribution of M_2 and M_5.

Fig. 7. The weight distribution of normal data and abnormal data in the user ACM2278 test datasets.

drops. Although other weights have small values, they still contribute a little to the model (-T1T2A1A2). In addition, the model detection accuracy in the table does not change, which may be because the deleted GNN layer weight value is too small and does not have much impact on the model. The above experiment shows that the distance feature (the similarity of nodes) is an important feature that contributes to the model's detection ability. M_i, $(i = 1, \ldots, 6)$ can indicate each feature's contribution of distance, time, and attention to the model's detection ability. The value of M_i, $(i = 1, \ldots, 6)$ is automatically calculated by the trained MEWRGNN model according to different input samples to ensure that the model can automatically evaluate the importance of different features.

To intuitively experience the 6 weights $(M_i,\ i = 1, \ldots, 6)$ that the MEWRGNN model automatically outputs based on the input samples. The test datasets of the user ACM2278 are input into the model to generate box plots with 6 weights under normal and abnormal activities, respectively. In box-plot (Fig. 7), the hollow dots represent the abnormal points of data. The green horizontal line in the box represents the median of the data distribution. The upper and lower edges of the box are the upper and lower quartiles of the data, respectively. The whisker edges extending from the upper and lower edges of the box respectively represent the maximum and minimum values in the datasets. M_i_normal, $(i = 1, \ldots, 6)$ represents the weight value generated by using the normal activity logs, and M_i_abnormal, $(i = 1, \ldots, 6)$ represents the weight value generated by using the abnormal activity logs. It can be seen from Fig. 7 that the weight range generated by the normal data is closer, and the deviation distance of the abnormal point is smaller, which shows that normal activity logs have a certain regularity and generally change in a certain range. The

weight distribution of abnormal data is wider, and the deviation distance of abnormal points is farther. This shows that the abnormal data has a certain degree of randomness.

E. Comparison With Other Baseline Methods

To compare the proposed approach with other baseline methods, we choose GNN's classic model GCN and other machine learning methods (k-nearest neighbors, Naive Bayes, Decision Trees, Multi-layered perceptron) to perform insider threat detection experiments with the same public datasets. Table XIV shows the averaged evaluation results using 5-fold cross-validation with replacement.

The obtained results clearly demonstrate that the comprehensive performance of the MEWRGNN model outperforms those of other detection methods. The MEWRGNN model has high detection capabilities on multiple normal and abnormal users. GCN and MEWRGNN are comparable in detecting users ACM2278, PLJ1771, and DNS1758. However, GCN is inferior to MEWRGNN in other user detection. The Decision Tree approach performs poorly in detecting users CMP2946 and CDE1846. One can see that the MEWRGNN model is superior to Multi-layered perceptron (2 hidden layers, 112 hidden units) developed based on classical neural networks. Although Multilayer Perceptron has excellent ability to identify malicious activities of most users, the accuracy of activity detection for users CDE1846 and CMP2946 are slightly lower. K-nearest neighbor method (The parameter K is set to 2, 3, and 5, respectively.) also has high accuracy in classifying most user activities. However, it is not sensitive to the activities of user RSR2681. The Naive Bayes method has low accuracy in identifying most user activities. Analyzing Table XIV, we can see that the MEWRGNN and GCN model has high recognition accuracy for the activities of multiple users. However, other baseline methods can only accurately recognize the activities of individual users. This shows that the MEWRGNN and GCN model combined with the graph structure can mine the correlation between activities, captures better features, and improves recognition accuracy. Moreover, MEWRGNN can improve the detection accuracy based on GCN by adding multi-class relations (time-based edges, distance-based edges, and attention-based edges).

Table XV shows the FPR of two methods to detect malicious behavior. FPR represents the rate at which malicious behavior is predicted to be normal behavior. We can see that MEWRGNN's malicious behavior detection performance for users ACM2278 and PLJ1771 are slightly lower than the method proposed by [15], but the overall detection performance is better than the one in [15]. Reference [15] uses a rule-based unsupervised method to implement detection. The malicious behaviors of users CMP2946 and MBG3183 are not very aggressive, resulting in relatively high false alarms on [15]. MEWRGNN uses GNN to learn the normal behavior characteristics of each user, which help decrease the false alarm rate to a certain extent.

Reference [14] used malicious users ACM2278, PLJ1771, CDE1846 and 500 other normal users to conduct evaluation experiments. We use the same malicious user and 500 normal users for comparative evaluation. The evaluation result shows that Area Under the Curve (AUC) obtained in Reference [14] is 0.901. The AUC of our proposed method reaches 0.996. When using all malicious users and 500 normal users, the AUC

TABLE XIV
COMPARISON RESULTS OF THE PRESENTED MODEL AND OTHER BASELINE METHODS

CMP2946	Accuracy	Precision	Recall	F1 score
MEWRGNN	**0.90334**	**0.91264**	**0.90334**	**0.89059**
GCN	0.87840	0.89460	0.87840	0.85628
k-nearest neighbors (k = 2)	0.84053	0.83289	0.84053	0.80941
k-nearest neighbors (k = 3)	0.82272	0.80242	0.82272	0.80410
k-nearest neighbors (k = 5)	0.84633	0.83798	0.84633	0.82072
Naive Bayes	0.64098	0.74706	0.64098	0.67370
Decision Trees	0.77149	0.77627	0.77149	0.77368
Multi-layered perceptron	0.80935	0.80285	0.80935	0.80547
CDE1846	Accuracy	Precision	Recall	F1 score
MEWRGNN	**0.89848**	**0.91115**	**0.89848**	**0.89188**
GCN	0.89439	0.90658	0.89439	0.88746
k-nearest neighbors (k = 2)	0.87328	0.88173	0.87328	0.86487
k-nearest neighbors (k = 3)	0.86170	0.86318	0.86170	0.85516
k-nearest neighbors (k = 5)	0.87601	0.88452	0.87601	0.86799
Naive Bayes	0.80653	0.81295	0.80652	0.78711
Decision Trees	0.81607	0.81656	0.81607	0.81578
Multi-layered perceptron	0.85488	0.85312	0.85488	0.85345
ACM2278	Accuracy	Precision	Recall	F1 score
MEWRGNN	**0.98728**	**0.98747**	**0.98728**	**0.98720**
GCN	**0.98728**	**0.98747**	**0.98728**	**0.98720**
k-nearest neighbors (k = 2)	0.91519	0.92311	0.91519	0.91134
k-nearest neighbors (k = 3)	0.93993	0.94268	0.93993	0.93839
k-nearest neighbors (k = 5)	0.95053	0.95264	0.95053	0.94941
Naive Bayes	0.84947	0.85868	0.84947	0.83893
Decision Trees	0.94700	0.94704	0.94700	0.94669
Multi-layered perceptron	0.97668	0.97669	0.97668	0.97667
PLJ1771	Accuracy	Precision	Recall	F1 score
MEWRGNN	**0.99476**	**0.99483**	**0.99476**	**0.99475**
GCN	**0.99476**	**0.99483**	**0.99476**	**0.99475**
k-nearest neighbors (k = 2)	0.86288	0.88855	0.86288	0.85543
k-nearest neighbors (k = 3)	0.91441	0.92539	0.91441	0.91196
k-nearest neighbors (k = 5)	0.91616	0.92681	0.91616	0.91378
Naive Bayes	0.81834	0.84895	0.81834	0.80551
Decision Trees	0.95633	0.95678	0.95633	0.95617
Multi-layered perceptron	0.98690	0.98716	0.98690	0.98688
DNS1758	Accuracy	Precision	Recall	F1 score
MEWRGNN	**1.00000**	**1.00000**	**1.00000**	**1.00000**
GCN	**1.00000**	**1.00000**	**1.00000**	**1.00000**
k-nearest neighbors (k = 2)	0.93403	0.93923	0.93403	0.92831
k-nearest neighbors (k = 3)	0.95473	0.95732	0.95473	0.95217
k-nearest neighbors (k = 5)	0.94956	0.95270	0.94956	0.94637
Naive Bayes	0.89952	0.89689	0.89952	0.89095
Decision Trees	0.98663	0.98665	0.98663	0.98655
Multi-layered perceptron	0.99914	0.99914	0.99914	0.99914
RSR2681	Accuracy	Precision	Recall	F1 score
MEWRGNN	**0.99545**	**0.99548**	**0.99545**	**0.99543**
GCN	0.99375	0.99381	0.99375	0.99379
k-nearest neighbors (k = 2)	0.79773	0.79967	0.79773	0.79830
k-nearest neighbors (k = 3)	0.76648	0.81271	0.76648	0.77818
k-nearest neighbors (k = 5)	0.75398	0.81052	0.75398	0.76723
Naive Bayes	0.90057	0.90373	0.90057	0.90108
Decision Trees	0.96931	0.96939	0.96932	0.96927
Multi-layered perceptron	0.96989	0.97025	0.96989	0.96964
QCD0386	Accuracy	Precision	Recall	F1 score
MEWRGNN	**0.99721**	**0.99722**	**0.99721**	**0.99721**
GCN	0.99675	0.99676	0.99675	0.99674
k-nearest neighbors (k = 2)	0.89559	0.90493	0.89559	0.88250
k-nearest neighbors (k = 3)	0.91926	0.92218	0.91926	0.91323
k-nearest neighbors (k = 5)	0.91926	0.92602	0.91926	0.91181
Naive Bayes	0.84084	0.84064	0.84084	0.80969
Decision Trees	0.98561	0.98586	0.98561	0.98568
Multi-layered perceptron	0.98144	0.98152	0.98144	0.98139
RJH0219	Accuracy	Precision	Recall	F1 score
MEWRGNN	**0.99732**	**0.99733**	**0.99731**	**0.99731**
GCN	0.99141	0.99149	0.99141	0.99138
k-nearest neighbors (k = 2)	0.91353	0.91660	0.91353	0.90838
k-nearest neighbors (k = 3)	0.91513	0.91378	0.91513	0.91306
k-nearest neighbors (k = 5)	0.92372	0.92382	0.92372	0.92062
Naive Bayes	0.83888	0.83594	0.83888	0.82023
Decision Trees	0.98443	0.98468	0.98443	0.98446
Multi-layered perceptron	0.97100	0.97094	0.97100	0.97092

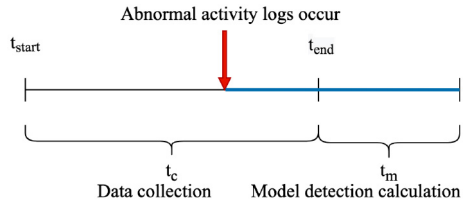


Fig. 8. The model detection time.

TABLE XV
COMPARISON OF THE PERFORMANCE OF MALICIOUS BEHAVIOR DETECTION BETWEEN MEWRGNN AND THE METHOD IN [15]

User	ACM2278	CMP2946	PLJ1771	CDE1846	MBG3183
The method in [15]					
FPR to detect all malicious instances	0.06%	6.97%	0.00%	0.32%	8.36%
MEWRGNN					
FPR to detect all malicious instances	0.62%	0.10%	0.03%	0.03%	0.10%

of our proposed method is 0.995. AUC is used to evaluate the model comprehensively. The higher the value, the better the performance. From the results above, the proposed method has better performance.

F. Performance Analysis

In an insider threat detection system, it is generally necessary to identify abnormal user activities in a timely manner. Therefore, the detection time is an important metric for evaluating insider threat detection systems. The detection time [51] is defined as:

$$t_d = t_f - t_o, (t_{start} \leq t_o \leq t_{end}) \tag{50}$$

where t_f is the moment when the insider threat is detected and t_o is the moment when the insider threat has appeared in the information system of the organization or enterprise. The detection time t_d denotes the length of time from when the insider threat enters the information system to when the threat is detected. t_{start} and t_{end} are the start and end time of collecting 30 activity Logs. t_f can be divided into two phases: data collection t_c and model detection t_m. Therefore, the detection time is equivalent to

$$t_d = t_c - t_o + t_m \tag{51}$$

Fig. 8 describes the time course of model detection. We can see that during the activity log collection, the insider threat may occur at any time between t_{start} and t_{end}. When $t_o = t_{start}$, the detection time is maximum $(t_d = t_c + t_m)$, and when $t_o = t_{end}$, the detection time is minimized $(t_d = t_m)$. Considering that insider threat operations generally occurs within a continuous period of time.

The detection delay of insider threats is mainly from collecting enough activity logs, while the time spent in the model detection phase is negligible. Table XVI presents the delay time for MEWRGNN and the method in [15] to detect each malicious behavior. Collecting 30 activity logs with varying times, MEWRGNN presents the arithmetic average of the detection delay time. We can see that MEWRGNN can detect various malicious behaviors relatively quickly. However, it takes more than three days for the one in [15] to detect the malicious behavior of CDE1846.

TABLE XVI
COMPARISON OF DETECTION DELAY TIME FOR EACH MALICIOUS BEHAVIOR BETWEEN MEWRGNN AND [15], (UNIT: DAY)

User	The method in [15]	MEWRGNN
ACM2278	0.22	0.07
CMP2946	0.74	0.06
PLJ1771	0.79	0.10
CDE1846	3.07	0.12
MBG2278	0.57	0.09

VI. CONCLUSION AND FUTURE WORK

A novel robust anomaly-based insider threat detection approach (i.e., MEWRGNN) using GNN model has been proposed. By using the combined graph neural networks, time series like user behavior logs are transformed into a graph for node feature construction and contextual features extraction. Multiple node feature extraction can characterize the contextual relationship of different user behavior over a period of time better. Anomaly-based insider threat detection process with the combined several GNNs is presented in detail. Then the MEWRGNN is evaluated upon the public known CERT dataset. The results show that the proposed model has strong robustness, and can achieve a high detection accuracy and better performance. The MEWRGNN model possesses certain interpretability as well, and the weight of each edge type can indicate the edge feature's influence on the model's ability.

For future work, we will study how to use the relationships among the internal personnel in the datasets better, to automatically build the graph structure while adding more features to the weight edge to find better abstract representation, to improve the detection accuracy of the model further. The hourly threat detection delay may still be slightly longer, and more real-time detection techniques need further study.

APPENDIX
THE DESCRIPTION OF EACH COLUMN IN THE LOG FORMAT

- "Date" field records the time of user activity.
- "User" field records the user who generated the activity.
- "PC" field records the host used by the user who generated the activity.
- "File_tree" field records the storage address of the mobile device.
- "Activity" field records the type of operation of the activity. For example, "Send" means sending e-mails and "WWW Visit" means browsing Webs.
- "TO" field records the recipient's email address.
- "CC" field records the email address where the carbon copy is sent.
- "BCC" field records the email address where a blind carbon copy is sent.

- "From" field records the sender's email address.
- "Size" field records the size of the email.
- "Attachments" field records the storage address of email attachments.
- "Content" field records the contents of emails, websites and files in a text format.
- "Filename" field records the storage address and type of the file accessed by the user.
- "To_removable_media" field records whether the file on the host is moved to a removable device, ("TRUE" is for the case that the file is moved to a removable device, while "FALSE" is for the opposite case).
- "from_removable_media" field records whether the file on the removable device is moved to the host, ("TRUE" for yes, while "FALSE" for no).
- "URL" field records the Web address that the user visits.

ACKNOWLEDGMENT

The authors would like to thank the Editor and anonymous reviewers for their helpful comments and suggestions.

REFERENCES

[1] G. Silowash, D. Cappelli, A. Moore, R. Trzeciak, T. J. Shimeall, and L. Flynn, "Common sense guide to mitigating insider threats (4th edition)," Dept. Softw. Eng. Inst., Carnegie Mellon Univ., Pittsburgh, PA, USA, Rep. CMU/SEI-2012-TR-012, Dec. 2012. [Online]. Available: https://resources.sei.cmu.edu/asset_files/TechnicalReport/2012_005_00_34033.pdf

[2] CERT Insider Threat Center. "Insider threat vulnerability assessment." Software Engineering Institute. Nov. 2015. [Online]. Available: https://resources.sei.cmu.edu/asset_files/Brochure/2015_015_001_51650.pdf

[3] "2018 insider threat report," Crowd Res. Partners, Rep., 2018. [Online]. Available: https://crowdresearchpartners.com/insider-threat-report/

[4] M. Collins et al., "Common sense guide to mitigating insider threats," Dept. Softw. Eng. Inst., Carnegie Mellon Univ., Pittsburgh, PA, USA, Rep. CMU/SEI-2016-TR-015, 2016. [Online]. Available: http://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=484738

[5] G. Yang, J. Ma, A. Yu, and Z. Meng, "Survey of insider threat detection," *J. Cyber Security*, vol. 1, no. 3, pp. 21–36, 2016.

[6] S. Yuan and X. Wu, "Deep learning for insider threat detection: Review, challenges and opportunities," 2020, *arXiv:2005.12433*.

[7] D. Cappelli, A. Moore, and R. Trzeciak, *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes* (Theft, Sabotage, Fraud). Upper Saddle River, NJ, USA: Addison-Wesley, Jan. 2012.

[8] I. Homoliak, F. Toffalini, J. Guarnizo, Y. Elovici, and M. Ochoa, "Insight into insiders and IT: A survey of insider threat taxonomies, analysis, modeling, and countermeasures," *ACM Comput. Surv.*, vol. 52, no. 2, pp. 1–40, 2019.

[9] T. Casey, "A field guide to insider threat," White Paper, Oct. 2015. [Online]. Available: https://www.researchgate.net/publication/324068663_A_Field_Guide_to_Insider_Threat_Understanding_Insider_Threat_Vectors_to_Further_Improve_Enterprise_Security_Strategies

[10] M. N. Al-Mhiqani, R. Ahmad, W. Mohamed, A. Hassan, and K. Hameed, "Cyber-security incidents: A review cases in cyber-physical systems," *Int. J. Adv. Comput. Sci. Appl.*, vol. 9, no. 1, pp. 499–508, 2018.

[11] M. N. Al-Mhiqani et al., "A review of insider threat detection: Classification, machine learning techniques, datasets, open challenges, and recommendations," *Appl. Sci.*, vol. 10, no. 15, pp. 1–41, 2020.

[12] A. Azaria, A. Richardson, S. Kraus, and V. S. Subrahmanian, "Behavioral analysis of insider threat: A survey and bootstrapped prediction in imbalanced data," *IEEE Trans. Comput. Soc. Syst.*, vol. 1, no. 2, pp. 135–155, Jun. 2014.

[13] D. C. Le, N. Zincir-Heywood, and M. I. Heywood, "Analyzing data granularity levels for insider threat detection using machine learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 1, pp. 30–44, Mar. 2020.

[14] L. Liu, C. Chen, J. Zhang, O. De Vel, and Y. Xiang, "Doc2vec-based insider threat detection through behaviour analysis of multi-source security logs," in *Proc. IEEE Int. Conf. Trust Security Privacy Comput. Commun.*, Guangzhou, China, 2020, pp. 301–309.

[15] D. C. Le and N. Zincir-Heywood, "Anomaly detection for insider threats using unsupervised ensembles," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1152–1164, Jun. 2021.

[16] D. Li, L. Yang, H. Zhang, X. Wang, L. Ma, and J. Xiao, "Image-based insider threat detection via geometric transformation," *Security Commun. Netw.*, vol. 2021, Sep. 2021, Art. no. 1777536.

[17] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, London, U.K., Nov. 2019, pp. 1777–1794.

[18] L. Liu, O. De Vel, Q.-L. Han, J. Zhang, and Y. Xiang, "Detecting and preventing cyber insider threats: A survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 2, pp. 1397–1417, 2nd Quart., 2018.

[19] F. L. Greitzer and R. E. Hohimer, "Modeling human behavior to anticipate insider attacks," *J. Strategic Security*, vol. 4, no. 2, pp. 25–48, 2011.

[20] O. Brdiczka et al., "Proactive insider threat detection through graph learning and psychological context," in *Proc. IEEE Symp. Security Privacy Workshops*, 2012, pp. 142–149.

[21] P. Parveen, Z. R. Weger, B. Thuraisingham, K. Hamlen, and L. Khan, "Supervised learning for insider threat detection using stream mining," in *Proc. IEEE Int. Conf. Tools Artif. Intell.*, Boca Raton, FL, USA, 2011, pp. 1032–1039.

[22] F. Yuan, Y. Cao, Y. Shang, Y. Liu, J. Tan, and B. Fang, "Insider threat detection with deep neural network," in *Proc. Int. Conf. Comput. Sci.*, vol. 10860. Cham, Switzerland, 2018, pp. 43–54.

[23] T. Hu, W. Niu, X. Zhang, X. Liu, J. Lu, and Y. Liu, "An insider threat detection approach based on mouse dynamics and deep learning," *Security Commun. Netw.*, vol. 2019, pp. 1–12, Feb. 2019.

[24] P. Chattopadhyay, L. Wang, and Y.-P. Tan, "Scenario-based insider threat detection from cyber activities," *IEEE Trans. Comput. Soc. Syst.*, vol. 5, no. 3, pp. 660–675, Sep. 2018.

[25] A. Tuor, S. Kaplan, B. Hutchinson, N. Nichols, and S. Robinson, "Deep learning for unsupervised insider threat detection in structured cybersecurity data streams," Oct. 2017, *arXiv:1710.00811*.

[26] B. Böse, B. Avasarala, S. Tirthapura, Y.-Y. Chung, and D. Steiner, "Detecting insider threats using RADISH: A system for real-time anomaly detection in heterogeneous data streams," *IEEE Syst. J.*, vol. 11, no. 2, pp. 471–482, Jun. 2017.

[27] P. Parveen and B. Thuraisingham, "Unsupervised incremental sequence learning for insider threat detection," in *Proc. IEEE Int. Conf. Intell. Security Inform.*, Washington, DC, USA, 2012, pp. 141–143.

[28] D. Nathani, J. Chauhan, C. Sharma, and M. Kaul, "Learning attention-based embeddings for relation prediction in knowledge graphs," 2019, *arXiv:1906.01195*.

[29] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.

[30] J. Jiang et al., "Anomaly detection with graph convolutional networks for insider threat and fraud detection," in *Proc. IEEE Mil. Commun. Conf.*, 2019, pp. 109–114.

[31] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *Proc. IEEE Security Privacy Workshops*, 2013, pp. 98–104.

[32] L. Zeigermann, "TIMESTAMP: Stata module to obtain a UNIX timestamp and the current time of a user-specified timezone," Dept. Economics, Boston College, Newton, MA, USA, Statistical Software Components Rep. S458182, 2016.

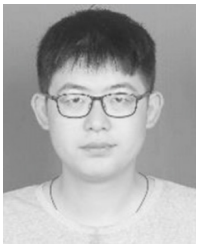
[33] "Pandas.factorize." 2022. [Online]. Available: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.factorize.html

[34] B. Shi, J. Zhao, and K. Xu, "A word2vec model for sentiment analysis of Weibo," in *Proc. Int. Conf. Serv. Syst. Serv. Manag.*, Shenzhen, China, 2019, pp. 1–6.

[35] J. Lilleberg, Y. Zhu, and Y. Zhang, "Support vector machines and Word2vec for text classification with semantic features," in *Proc. Int. Conf. Cogn. Inform. Cogn. Comput.*, Beijing, China, 2015, pp. 136–140.

[36] Y.-A. Chung, C.-C. Wu, C.-H. Shen, H.-Y. Lee, and L.-S. Lee, "Audio Word2Vec: Unsupervised learning of audio segment representations using sequence-to-sequence Autoencoder," in *Proc. INTERSPEECH*, San Francisco, CA, USA, Aug. 2016, pp. 765–769.

[37] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 3111–3119.
[38] R. Řehůřek and P. Sojka, "Software framework for topic modelling with large corpora," in *Proc. LREC Workshop New Challenges NLP Frameworks*, Valletta, Malta, May 2010, pp. 45–50. [Online]. Available: http://is.muni.cz/publication/884893/en
[39] T. Mikolov, E. Grave, P. Bojanowski, C. Puhrsch, and A. Joulin, "Advances in pre-training distributed word representations," 2018, *arXiv:1712.09405*.
[40] "Arithmetic average advantages and disadvantages." Macroption. Accessed: Jun. 21, 2022. [Online]. Available: https://www.macroption.com/arithmetic-average-advantages-disadvantages/#advantage-1-fast-and-easy-to-calculate
[41] I. H. Witten, E. Frank, and M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques*. Burlington, MA, USA: Morgan Kaufmann, 2011, pp. 131–137.
[42] D. Li, J. Lin, T. F. D. A. Bissyandé, J. Klein, and Y. Le Traon, "Extracting statistical graph features for accurate and efficient time series classification," in *Proc. Int. Conf. Extending Database Technol.*, Vienna, Austria, 2018, pp. 1–12.
[43] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," Sep. 2016, *arXiv:1609.02907*.
[44] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. Eur. Semantic Web Conf.*, vol. 10843. Cham, Switzerland, 2018, pp. 593–607.
[45] J. Xiao, L. Yang, F. Zhong, H. Chen, and X. Li, "Robust anomaly-based intrusion detection system for in-vehicle network by graph neural network framework," *Appl. Intell.*, to be published.
[46] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
[47] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," Sep. 2020, *arXiv:2009.02040*.
[48] Z. Chen, D. Chen, Z. Yuan, X. Cheng, and X. Zhang, "Learning graph structures with transformer for multivariate time series anomaly detection in IoT," Apr. 2021, *arXiv:2104.03466*.
[49] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," Dec. 2014, *arXiv:1412.6980*.
[50] L. Van Der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 86, pp. 2579–2605, 2008.
[51] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198.



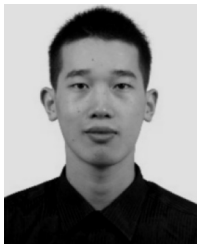
Junchao Xiao received the M.S. degree from Shanghai Maritime University, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Systems Science and Engineering, Sun Yat-sen University. His research interests include deep learning, network security, and intrusion detection system.

Lin Yang received the M.S. degree in automation and the Ph.D. degree in communication and electronic system from the National University of Defense Technology, Changsha, Hunan, in 1995 and 1999, respectively. Since 2005, he has been a Senior Engineer with China Electronic Equipment and System Engineering Corporation. His research interests include computer security, information system security, network security, trusted computing, security protocol analysis, and big-data security.

Fuli Zhong received the B.S. degree in electronic information engineering and the Ph.D. degree in navigation, guidance and control from the University of Electronic Science and Technology of China, Chengdu, China, in 2011 and 2019, respectively. He is currently a Postdoctoral Fellow with the School of Systems Science and Engineering, Sun Yat-sen University. His research interests include cybersecurity and resilience, machine learning, cyber-physical system, prognostics and health management, full-duplex wireless communications, signal processing, optimization, complex nonlinear dynamic systems, state estimation, and identification and control of dynamic systems.

Xiaolei Wang received the M.S. and Ph.D. degrees in computer science and technology from the College of Computer, National University of Defense Technology, China, in 2014 and 2019, respectively. His research interests include malware analysis, program analysis, and software security and defense.

Hongbo Chen received the Ph.D. degree from the Harbin Institute of Technology in 2007. He is currently a Professor with the School of Systems Science and Engineering, Sun Yat-sen University, where he is the Dean with the School of Systems Science and Engineering. His main research interests include modeling, simulation and analysis of complex systems, intelligent unmanned systems, and design of aerospace vehicle.



Dongyang Li is currently pursuing the Ph.D. degree with the Army Engineering University of PLA. His research interests include insider threat detection and network measurement and monitor.